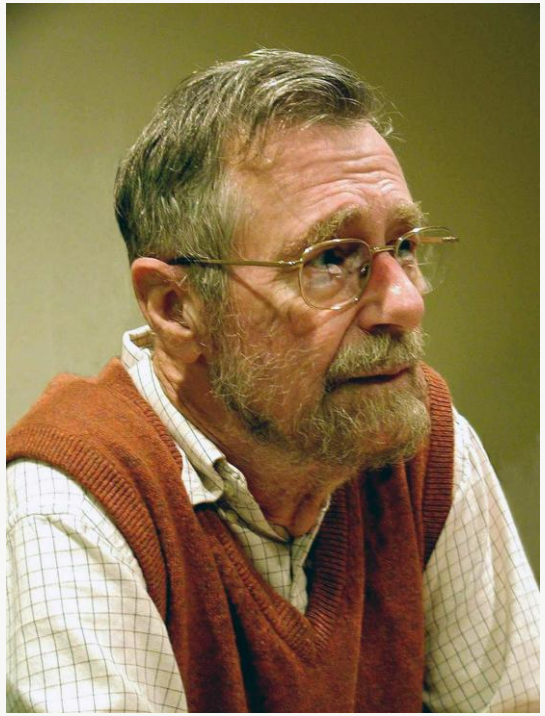
Az algoritmikus problémamegoldás alapjai

Algoritmusok és helyesség

Problémaleírás, algoritmusterv
és a helyesség indoklása



Edsger W. Dijkstra

Mikor fogadnánk el egy megoldást?

1. A program lefordul, és egy példára jó eredményt ad.
2. Minden megengedett bemenetre jó eredményt ad, ha befejeződik.
3. Minden megengedett bemenetre befejeződik, és jó eredményt ad.

A helyesség vizsgálata a pontos feladatmegadáshoz kapcsolódik.

A probléma a megoldandó feladat

Mit kapunk?

A lehetséges bemenetek
és a rájuk vonatkozó feltételek.

Mit várunk?

A bemenethez tartozó
elfogadható eredmény.

A probléma megadása rögzíti, mit tekintünk jó válasznak.

Ugyanaz a feladat több konkrét bemenettel

Fogalom	Példa
Probléma	Két egész szám maximumának meghatározása
Egy példány	a = 7, b = 3
Másik példány	a = −2, b = −5
Az elvárt eredmények	7, illetve −2

Az algoritmus a teljes megengedett bemeneti körre ad eljárást.

A specifikáció alapvető részei

Bemenet

Milyen adatokat kap az eljárás?

Feltételek

Mely bemenetek megengedettek?

Eredmény

Mit kell teljesítenie a kimenetnek?

A specifikáció a helyesség vizsgálatának alapja.

Mit jelent az, hogy számítsa ki az átlagot?

Hiányzó részletek

Hány adat van?
Lehet nulla darab adat?
Milyen átlagot kérünk?

Pontosított változat

n > 0 darab valós szám
számtani átlaga:
az összeg osztva n-nel.

A tisztázatlan feltételek eltérő megoldásokhoz vezethetnek.

Két egész szám maximumának specifikációja

Rész	Pontos megadás
Bemenet	a és b egész szám
Kimenet	m egész szám
Elvárt kapcsolat	m ≥ a és m ≥ b, továbbá m = a vagy m = b
Egyenlőség esetén	Ha a = b, akkor m a közös érték

A kimenet az egyik bemeneti érték, és egyiknél sem kisebb.

Az algoritmus a megoldás pontos lépéssora

Végrehajtható lépések

A megengedett bemenetből
az elvárt eredményhez vezető eljárás.

Ebben az órában véges számítási feladatokat vizsgálunk.

A vizsgált algoritmusok jellemzői

Tulajdonság	Jelentés
Pontosság	A lépések és a következő lépés szabálya rögzített
Végrehajthatóság	Minden alaplépés elvégezhető
Végesség	Minden megengedett bemenetre véget ér
Általánosság	A feladat megengedett példányaira alkalmazható

A mai példák determinisztikusak: azonos állapotból azonos lépés következik.

A lépéssor és a programnyelvi megvalósítás

Algoritmus

A megoldás logikája.
Leírható pszeudokóddal
vagy folyamatábrával.

Program

Konkrét nyelven leírt utasítások
és a végrehajtáshoz szükséges
adatkezelés.

Ugyanazt az algoritmust több programnyelven is megírhatjuk.

A megoldás kidolgozásának lépései

1. A bemenet, a kimenet és a feltételek tisztázása.
2. Egyszerű példák és szélső esetek elemzése.
3. Az algoritmus megtervezése és leírása.
4. A helyesség indoklása, majd a program ellenőrzése.

A hibás feladatértelmezést a kódolás önmagában nem javítja ki.

A maximum példájának fontos próbái

Bemenet	Elvárt eredmény	Mit vizsgál?
7 és 3	7	Az első érték nagyobb
3 és 7	7	A második érték nagyobb
4 és 4	4	Egyenlő értékek
−2 és −5	−2	Két negatív érték

A próbák a specifikáció különböző eseteit fedik le.

A lépések szervezésének alapjai

Szekvencia

A lépések egymás után következnek.

Elágazás

Egy feltétel alapján választunk végrehajtási ágat.

Ciklus

Egy lépéssort ismételten végrehajtunk.

A leírási eszközök ezeket a szerkezeteket is kifejezik.

Ugyanaz a logika több leírásban

Eszköz	Fő sajátosság
Szöveges leírás	Köznyelven, pontos sorrendben megadott lépések
Folyamatábra	Jelek és irányított kapcsolatok mutatják a vezérlést
Pszeudokód	Tömör, programhoz hasonló, nyelvfüggetlen jelölés

A különböző leírásoknak ugyanazt a működést kell megadniuk.

A maximum meghatározása szövegesen

1. Olvassa be a és b értékét.
2. Ha a ≥ b, legyen m értéke a.
3. Egyébként legyen m értéke b.
4. Írja ki m értékét.

A feltétel igaz és hamis esetének művelete is szerepel.

A használt jelölések

Jelölés	Jelentés
BE / KI	Adat átvétele / eredmény közlése
x ← kifejezés	Értékadás x változónak
HA … AKKOR … KÜLÖNBEN	Feltételes elágazás
AMÍG feltétel	Ismétlés, amíg a feltétel igaz

Az értékadás és az egyenlőségvizsgálat eltérő művelet.

A maximum meghatározása pszeudokóddal

```
BE a, b
HA a >= b AKKOR
  m ← a
KÜLÖNBEN
  m ← b
ELÁGAZÁS VÉGE
KI m
```

A két ág közül
pontosan egy fut le.

a = b esetén
m a közös érték.

A logikai működés megegyezik a szöveges leírással.

A folyamatábra alapjelei



A végrehajtás kezdete vagy vége

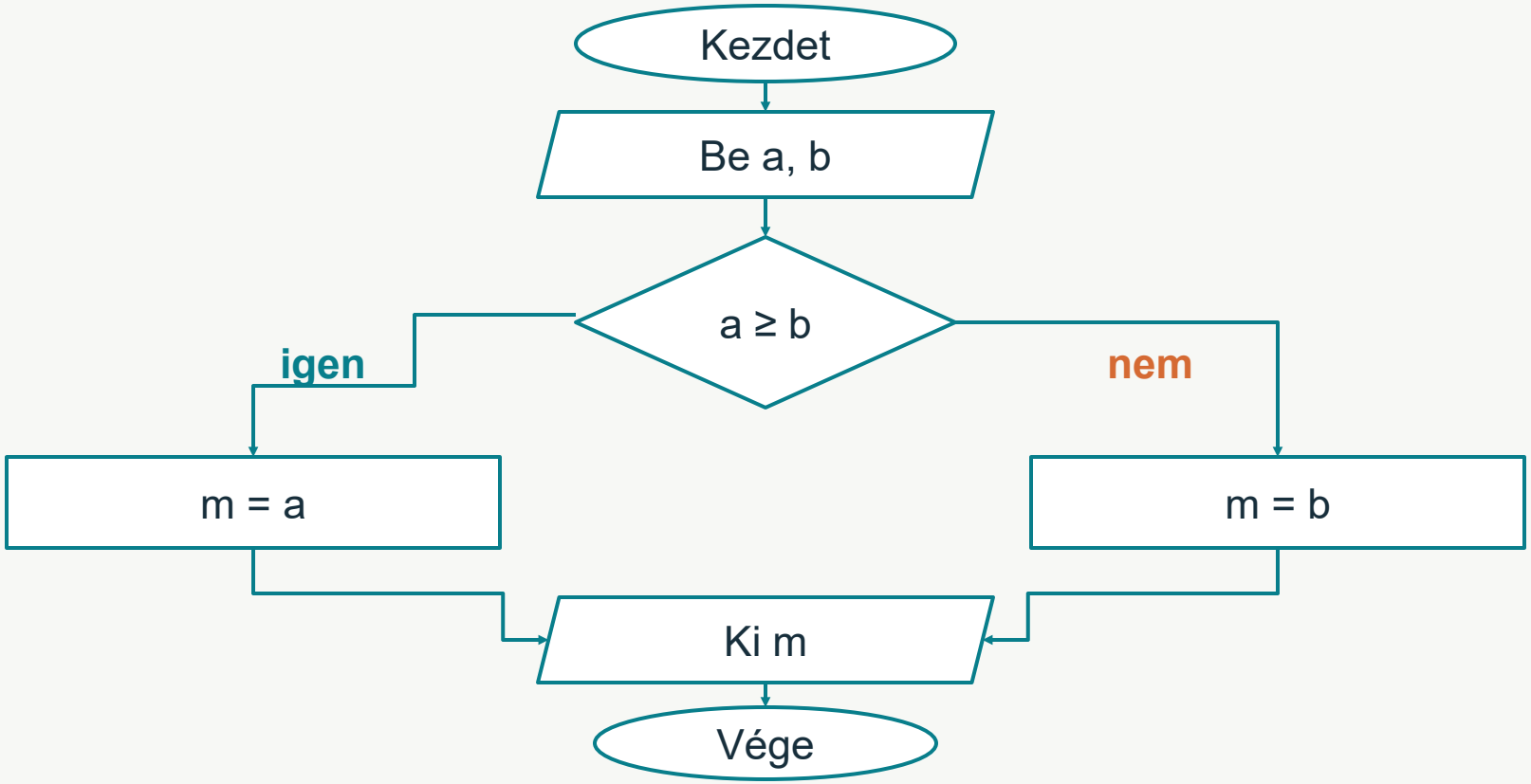
Például értékadás vagy számítás

Bemenet átvétele vagy eredmény közlése

Igaz és hamis ág közötti választás

Az irányított összekötő nyilak a következő lépést mutatják.

A maximumfeladat végrehajtási útvonalai



Rombusz: feltétel Téglalap: értékadás Paralelogramma: be és ki

A leírási forma választása

Szempont	Szöveg	Folyamatábra	Pszeudokód
Kezdő magyarázat	Közvetlen	Jól követhető	Jelölést igényel
Elágazások	Szavakkal	Külön ágakkal	HA szerkezettel
Hosszabb algoritmus	Terjedelmes lehet	Nagy ábra lehet	Általában tömör

Az átíráskor minden feltételnek és műveletnek meg kell maradnia.

Egy konkrét futás állapotai

Lépés	a	b	m
Bemenet után	−2	−5	Még nincs értéke
Feltétel: a ≥ b	−2	−5	Az igaz ág következik
Értékadás után	−2	−5	−2
Eredmény	−2	−5	−2

A nyomkövetés egy konkrét bemenet végrehajtását mutatja.

Tesztelés és helyességi érvelés

Tesztelés

Konkrét bemeneteken ellenőrizzük a működést.
Egy hibás eredmény ellenpélda.

Helyességi érvelés

A specifikációból és a lépésekből indokoljuk az eredményt a megengedett bemenetekre.

Néhány sikeres próba általános esetben nem bizonyítja a helyességet.

A feladat és a megoldás megkülönböztetése

1. Mi hiányzik a „számítsa ki az átlagot” feladatból?
2. Elég-e a maximumra az m ≥ a és m ≥ b feltétel?
3. Miért fontos a két negatív számot tartalmazó próba?

A közös kérdések válaszai

1. Rögzíteni kell az adatokat és az üres bemenet kezelését.
2. m-nek a vagy b értékével is egyeznie kell.
3. A nulla nem megfelelő maximum két negatív számra.

Az ellenőrzéshez előbb pontosan meg kell adni az elvárt működést.

A helyesség a specifikációhoz viszonyít

Megfelel a feladatnak?

Ugyanaz a program eltérő specifikációk
mellett eltérő megítélést kaphat.

Példa: a két érték közül mindig az elsőt választani csak külön feltétellel jó.

A számítás megkezdésekor elvárt állapot

P: előfeltétel

Megadja, milyen kezdőállapotokból vállaljuk az elvárt eredményt.

Példa

Osztásnál az osztó nem nulla.
Nem üres sorozat maximumánál legalább egy adat van.

A megengedett bemeneti tartomány az előfeltétel része lehet.

A befejezéskor elvárt állapot

Q: utófeltétel

Az elvárt kapcsolat a bemenet
és az eredmény között.

Maximum példa

m ≥ a és m ≥ b
és m a vagy b értéke.
A bemeneteket nem módosítjuk.

Az utófeltételt a normális befejezés állapotában vizsgáljuk.

Előfeltétel, programrész és utófeltétel

$$\{P\}\ C\ \{Q\}$$

P: előfeltétel C: programrész
Q: utófeltétel

Részleges helyesség: ha P igaz, és C befejeződik, akkor Q igaz.

Egy értékadás helyessége

```
{ x = 4 }
x ← x + 1
{ x = 5 }
```

A kezdőállapotban x = 4.
Az értékadás után x = 5.

Egy végrehajtható alaplépés befejeződik.

Az állítás a megadott előfeltétel mellett igaz.

A befejeződő futások eredménye helyes

Ha befejeződik, jó eredményt ad

Minden P-t teljesítő kezdőállapotból
a befejeződő futás végén Q teljesül.

A részleges helyesség önmagában nem garantálja a befejezést.

Jó eredmény, amikor az eljárás befejeződik

```
BE x
AMÍG x != 0
  SEMMIT NEM VÁLTOZTATUNK
CIKLUS VÉGE
y ← 0
KI y
```

P: x egész szám.
Q: y = 0.

x = 0: befejeződik.
x ≠ 0: nem ér véget.

A Q utófeltétel minden befejeződő futásra igaz.

Helyes eredmény és biztos befejezés

Részleges helyesség

Minden megengedett, befejeződő futás végén teljesül az utófeltétel.

Terminálás

Minden megengedett bemenetre véges számú lépés után befejeződik a számítás.

A két tulajdonság együtt adja a teljes helyességet.

A helyességi állítások különbsége

Jelölt eljárás	Részlegesen helyes?	Teljesen helyes?
Mindig befejeződik és Q igaz	Igen	Igen
Néha nem ér véget, egyébként Q igaz	Igen	Nem
Egy megengedett futás hibás eredménnyel végződik	Nem	Nem

Minden sor ugyanahhoz a rögzített P és Q feltételhez viszonyít.

Öt perc szünet

5 perc

Utána: a terminálás és a ciklusinvariáns
közös vizsgálata.

A véges futás lehet hosszú is

Terminálás

Véges számú lépés után
véget ér a vizsgált futás.

Hatékonyság

Mennyi időt és memóriát
igényel a megoldás
a bemenet méretéhez képest?

A teljes helyesség önmagában nem ad gyorsasági garanciát.

Egész számláló csökkentése

```
P: n >= 0 egész
k ← n
AMÍG k > 0
  k ← k - 1
CIKLUS VÉGE
```

n = 3 esetén:
k: 3, 2, 1, 0

A k nemnegatív egész.
Minden körben 1-gyel csökken.

A ciklus n ismétlés után befejeződik.

A szigorú csökkenés önmagában kevés

```
x ← 1
AMÍG x > 0
  x ← x / 2
CIKLUS VÉGE
```

Pontos valós aritmetikában:
1, 1/2, 1/4, 1/8, …

Minden érték pozitív.
A ciklus nem ér véget.

A terminálási mérőszámhoz alkalmas, jól megalapozott tartomány kell.

Egy állítás az ismétlődően elért állapotokról

A kijelölt ciklusponton mindig igaz

A ciklus első feltételvizsgálata előtt,
majd minden újabb feltételvizsgálat előtt.

A feltétel állandó igazsága mellett a változók értéke változhat.

A ciklusmag közben átmenetileg sérülhet

```
összeg ← 0
i ← 1
// I itt igaz
AMÍG i <= n
  összeg ← összeg + i
  i ← i + 1
  // I újra igaz
CIKLUS VÉGE
```

I: összeg = 1 + … + (i−1)

A két értékadás között ez az állítás átmenetileg nem feltétlenül igaz.

A megőrzést a teljes ciklusmagra kell igazolni.

Az invariáns három feladata

Inicializálás

A ciklus előtt létrehozzuk az állítást.

Megőrzés

Egy teljes ciklusmag ismét létrehozza.

Kilépés

A hamis ciklusfeltétellel együtt megadja az eredményt.

A teljes helyességhez ezen felül a terminálást is igazoljuk.

Az 1-től n-ig összegzés specifikációja

Rész	Megadás
Bemenet	n egész szám, amelyet nem módosítunk
P: előfeltétel	$n \geq 0$
Kimenet	összeg egész szám
Q: utófeltétel	összeg = $1 + 2 + \ldots + n = n(n + 1) / 2$

n = 0 esetén az üres összeg értéke 0.

A vizsgált lépéssor

```
BE n
összeg ← 0
i ← 1
AMÍG i <= n
  összeg ← összeg + i
  i ← i + 1
CIKLUS VÉGE
KI összeg
```

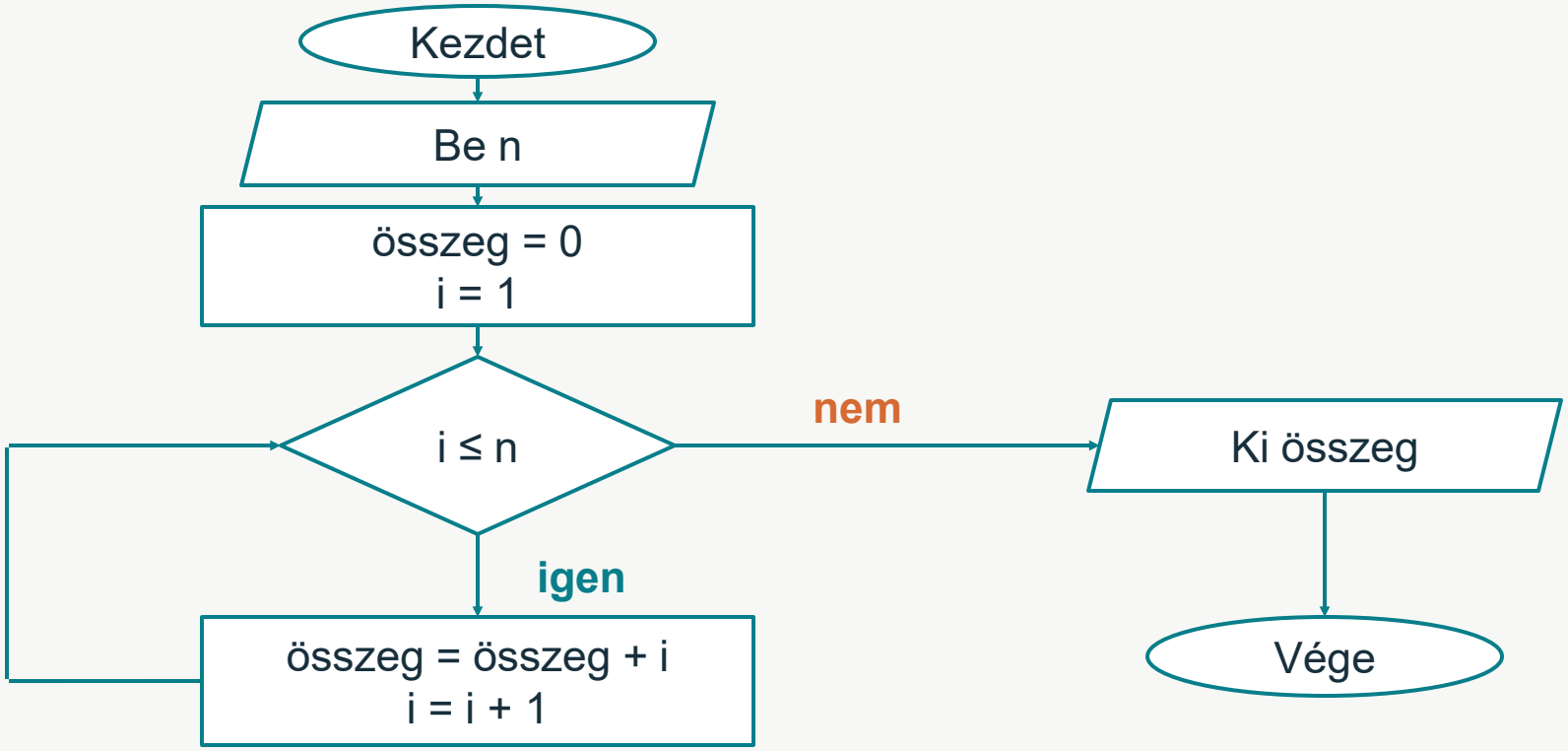
i: a következő hozzáadandó szám.

összeg: az eddig feldolgozott számok összege.

A ciklusmag mindkét értékadása szükséges.

A ciklusfej az invariáns ellenőrzési pontja

I: az első i − 1 szám összege az összeg változóban



Előfeltétel: n ≥ 0 egész

Összegzés n = 4 esetén, a ciklusfejben

i	összeg	Feldolgozott számok	Hátralévő körök
1	0	Még nincs	4
2	1	1	3
3	3	1, 2	2
4	6	1, 2, 3	1
5	10	1, 2, 3, 4	0

A táblázat minden sora azonos programhelyhez tartozik.

A feldolgozott rész pontos leírása

$$\text{összeg} = 1 + 2 + \ldots + (i - 1)$$

Továbbá: $1 \leq i \leq n + 1$
és $n \geq 0$, értéke változatlan.

A bal oldalon tárolt érték az első i − 1 szám összege.

A kezdőértékek létrehozzák az invariánst

Beállítás

összeg ← 0
i ← 1

Ellenőrzés

Az első $i - 1 = 0$ szám összege 0.
$1 \le 1 \le n + 1$, mert $n \ge 0$.

Az invariáns már az első feltételvizsgálat előtt igaz.

Egy teljes ciklusmag után újra igaz I

```
A kör elején i = k
összeg = 1 + … + (k−1)

összeg ← összeg + k
i ← k + 1

Ekkor összeg = 1 + … + k
```

Az új i értéke k + 1.

Az új összeg az első
(új i − 1) szám összege.

A feltételből k ≤ n.

Az új i legfeljebb n + 1, ezért a határfeltétel is megmarad.

Az invariánsból megkapjuk az utófeltételt

A kilépéskor

A feltétel hamis: $i > n$.
Az invariáns szerint $i \leq n + 1$.
Egész i mellett ezért $i = n + 1$.

Behelyettesítés

összeg = $1 + \ldots + (i - 1)$
$= 1 + \ldots + n$.
Ez az elvárt Q állítás.

Ez az érvelés a befejeződő futás eredményét igazolja.

A hátralévő ismétlések száma csökken

$$V = n - i + 1$$

A ciklusfejben V nemnegatív egész.
Minden lefutó ciklusmag 1-gyel csökkenti.

V = 0 esetén i = n + 1, ezért a ciklusfeltétel hamis.

Az összegző algoritmus teljes helyessége

Feladat	Eredmény
Inicializálás	Az I állítás igaz a ciklus előtt
Megőrzés	I és az igaz feltétel után a ciklusmag újra I-t ad
Kilépés	I és a hamis feltétel együtt Q-t ad
Terminálás	V nemnegatív egész, és minden körben csökken

P teljesülése esetén a számítás befejeződik, és Q teljesül.

Az i < n feltétel kihagyja az utolsó tagot

```
összeg ← 0
i ← 1
AMÍG i < n
  összeg ← összeg + i
  i ← i + 1
CIKLUS VÉGE
```

n = 4 esetén:
összeg = 1 + 2 + 3 = 6

Q szerint 10 kellene.

A futás befejeződik, de a megadott feladatra hibás.

Az elmaradó léptetés két hibát okoz

```
összeg ← 0
i ← 1
AMÍG i <= n
  összeg ← összeg + i
  // az i növelése hiányzik
CIKLUS VÉGE
```

n = 4 esetén i mindig 1.
A feltétel igaz marad.

Az összeg változik,
a feldolgozást nem követi i.

A fő példa invariánsa sérül, és a terminálási érv is elvész.

Az n = 0 bemenet is a bizonyítás része

Kezdőállapot

összeg = 0
i = 1
Az 1 ≤ 0 feltétel hamis.

Következmény

A ciklusmag nem fut.
Az eredmény 0.
Az üres összeg is 0.

A helyességi indoklás a nulla ismétléses esetre is érvényes.

A matematikai terv gépi megvalósítása

```
static long Osszeg(int n)
{
  long osszeg = 0;
  for (int i = 1; i <= n; i++)
    osszeg += i;
  return osszeg;
}
```

Előfeltétel a kódhoz:
$0 \le n \le 1\,000\,000$.

Az összeg long típusban,
a számláló int típusban
ezen a tartományon elfér.

A típusok tartománya a program helyességi feltételeihez tartozik.

A visszaszámlálás helyességének indoklása

```
P: n >= 0 egész
k ← n
darab ← 0
AMÍG k > 0
  k ← k - 1
  darab ← darab + 1
CIKLUS VÉGE
```

Cél: a végén darab = n.

Mi lehet az invariáns?
Mi csökken minden körben?
Miért jó az n = 0 eset?

A ciklusfejben érvényes kapcsolatot keresse.

A darab + k = n kapcsolat megmarad

Lépés	Indoklás
Kezdet	darab + k = 0 + n = n, k ≥ 0
Megőrzés	k 1-gyel csökken, darab 1-gyel nő
Kilépés	k = 0, ezért darab = n
Terminálás	A k nemnegatív egész, és minden körben csökken

Az invariáns és a terminálási mérőszám eltérő szerepet kap.

A helyesség legfontosabb különbségei

1. Mit garantál a részleges helyesség a befejezésről?
2. Mely programhelyen kell igaznak lennie az invariánsnak?
3. Miért alkalmas a nemnegatív egész, csökkenő mérőszám?

A 7. hét kulcsfogalmai

Fogalom	Szerep
Specifikáció	Megadja az elvárt működést
P és Q	Az indulás és a befejezés feltételei
Ciklusinvariáns	Összeköti a ciklus állapotait az eredménnyel
Terminálási mérőszám	A befejezés indoklását segíti
Teljes helyesség	Részleges helyesség és terminálás együtt

A jó eredményt és a befejezést egyaránt indokolni kell.

Fogalmak és formális helyesség

NIST: az algoritmus fogalma

https://xlinux.nist.gov/dads/HTML/algorithm.html

Software Foundations: Hoare-logika

https://softwarefoundations.cis.upenn.edu/plf-current/Hoare.html

University of Strathclyde: részleges és teljes helyesség

https://personal.cis.strath.ac.uk/robert.atkey/cs208/specify-verify.html

Instituto Superior Técnico: specifikáció és Hoare-hármasok

https://web.tecnico.ulisboa.pt/~joaofernandoferreira/2021/SoftSpec/02-hoare.html

Ciklusinvariáns és gépi megvalósítás

Cornell University: invariáns és terminálás

https://courses.cs.cornell.edu/cs2112/2020fa/lectures/loopinv/index.html

Microsoft Learn: C# egész típusok tartományai

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/integral-numeric-types

University of Texas at Austin: Dijkstra munkássága

https://www.cs.utexas.edu/~EWD/CSobit.html

Edsger W. Dijkstra portréja

Szerző: Hamilton Richards

CC BY-SA 3.0, Wikimedia Commons

https://commons.wikimedia.org/wiki/File:Edsger_Wybe_Dijkstra.jpg

Licenc

Az eredeti kép arányainak és tartalmának megőrzésével.

https://creativecommons.org/licenses/by-sa/3.0/